

IT INVENTORY SYSTEM - KOD DOKUMANI

Dosya: main.py

```
import sys, os, logging
from flask import Flask, jsonify, send_from_directory, send_file, request
from flask_cors import CORS
from dotenv import load_dotenv

load_dotenv()
BASE_DIR = os.path.abspath(os.path.dirname(__file__))

# Log klasörü kontrolü
if not os.path.exists(os.path.join(BASE_DIR, 'logs')):
    os.makedirs(os.path.join(BASE_DIR, 'logs'), exist_ok=True)

app = Flask(__name__)
app.secret_key = os.getenv('SECRET_KEY', 'KD_SECRET_2026')
CORS(app)

# --- STATİK DOSYA SERVİSİ (GÖRSEL VE STİL SORUNU ÇÖZÜMÜ) ---

@app.route('/')
def index():
    # Ana dizindeki index.html'i gönderir
    return send_file(os.path.join(BASE_DIR, 'index.html'))

@app.route('/<path:path>')
def catch_all(path):
    # Eğer istenen dosya ana dizinde veya bir alt klasörde (logo, frontend vb.) varsa gönderir
    file_path = os.path.join(BASE_DIR, path)
    if os.path.exists(file_path):
        return send_from_directory(BASE_DIR, path)
    # Eğer dosya bulunamazsa index'e yönlendirmez (hata almamak için)
    return f"Dosya bulunamadı: {path}", 404

# --- API VE DİĞER MODÜLLER ---
from core.database_sql import get_db_connection, init_db
from modules.inventory_manager import inventory_manager_bp
from modules.printer_manager import printer_manager_bp

app.register_blueprint(inventory_manager_bp, url_prefix='/api/inventory')
app.register_blueprint(printer_manager_bp, url_prefix='/api/printers')

@app.route('/api/inventory/search', methods=['GET'])
def global_search():
    q = request.args.get('q', '').upper()
    conn = get_db_connection()
    try:
        sql = "SELECT i.*, l.mahal_adi as mahal_adi_sql FROM inventory i LEFT JOIN locations l ON i.mahal_id = l.mahal_id"
        if q:
            sql += " WHERE UPPER(i.pc_no) LIKE ? OR UPPER(i.ip) LIKE ? OR UPPER(i.seri_no) LIKE ? OR UPPER(l.mahal_adi) LIKE ?"
            params = (f'%{q}%', f'%{q}%', f'%{q}%', f'%{q}%')
            results = conn.execute(sql, params).fetchall()
        else:
            results = conn.execute(sql).fetchall()
        return jsonify([dict(row) for row in results])
    finally:
        conn.close()

@app.route('/api/sync/all', methods=['POST'])
def sync_all():
    from core.sync_manager import SyncManager
    sync_mgr = SyncManager(BASE_DIR)
```

```

try:
    stats = sync_mgr.sync_inventory()
    return jsonify({"success": True, "details": stats})
except Exception as e:
    return jsonify({"success": False, "error": str(e)}), 500

if __name__ == '__main__':
    init_db()
    from waitress import serve
    print(f"Sistem Baslatiliyor... Dizin: {BASE_DIR}")
    serve(app, host='0.0.0.0', port=5000)

```

Dosya: core/sync_manager.py

```

import os
import pandas as pd
import logging
import traceback
from core.database_sql import get_db_connection
from core.utils import _clean, _norm_key

# 1. LOG YAPILANDIRMASI
logger = logging.getLogger('SyncDebug')
logger.setLevel(logging.DEBUG)
log_dir = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'logs')
try:
    if not os.path.exists(log_dir): os.makedirs(log_dir, exist_ok=True)
    handler = logging.FileHandler(os.path.join(log_dir, 'sync_debug.log'), encoding='utf-8', mode='w')
    handler.setFormatter(logging.Formatter('%(asctime)s [% (levelname)s] %(message)s'))
    logger.addHandler(handler)
except Exception:
    console_handler = logging.StreamHandler()
    logger.addHandler(console_handler)

class SyncManager:
    def __init__(self, base_dir):
        self.base_dir = base_dir
        # KURAL: Tüm Excel kaynaklar ana_database klasörü altında aranacak
        self.db_folder = os.path.join(self.base_dir, 'database', 'ana_database')

        self.mapping_rules = {
            'pc_no': ['pc no', 'cihaz no', 'kayitli cihaz no', 'no', 'id'],
            'seri_no': ['seri no', 'pc seri', 'seri numarasi', 'seri'],
            'monitor_seri': ['monitor seri', 'monitor no', 'monitor'],
            'mahal_id': ['mahal kodu', 'birim kodu', 'mahal', 'kod'],
            'ip': ['ip', 'ip adresi', 'network ip', 'ip address'],
            'mac': ['mac', 'mac adresi', 'ethernet'],
            'rdp': ['rdp', 'is_rdp', 'uzaktan baglanti', 'rdp durumu'],
            'depo': ['depo', 'depoda', 'is_depo'],
            'arizali': ['arizali', 'ariza', 'is_arizali'],
            'mahalsiz': ['mahalsiz', 'kayip', 'is_mahalsiz'],
            'bekliyor': ['kurulum bekliyor', 'bekleyen', 'is_kurulum_bekliyor', 'bekliyor'],
            'yazicilar': ['bagli yazicilar', 'yazicilar', 'yazici'],
            'phone': ['phone', 'telefon', 'tel', 'dahili'],
            'aciklama': ['aciklama', 'not', 'bilgi']
        }

    def _get_col_with_log(self, df_cols, key, sheet_name):
        variants = self.mapping_rules.get(key, [])
        for v in variants:
            if v in df_cols:
                logger.debug(f"[{sheet_name}] MAPPING: '{v}' -> '{key}'")
                return v
        return None

    def sync_inventory(self):

```

```

logger.info("=== ENVANTER SENKRONIZASYONU BASLATILDI ===")
path = os.path.join(self.db_folder, 'envanter.xlsx')

if not os.path.exists(path):
    logger.error(f"Excel bulunamadi: {path}")
    return {"error": f"Excel bulunamadi: {path}"}

tabs = {'BILGISAYAR': 'PC', 'SIRAMATIK': 'SIRAMATIK', 'TABLET': 'TABLET'}
conn = get_db_connection()
stats = {"PC": 0, "SIRAMATIK": 0, "TABLET": 0, "total": 0}

try:
    xl = pd.ExcelFile(path)
    for sheet in xl.sheet_names:
        s_norm = _norm_key(sheet)
        dev_type = next((v for k, v in tabs.items() if k in s_norm), None)
        if not dev_type: continue

        logger.info(f"--- Sekme Isleniyor: {sheet} ---")
        df = xl.parse(sheet)
        df.columns = df.columns.str.strip().str.lower()
        df_cols = list(df.columns)

        for index, row in df.iterrows():
            try:
                def _val(k):
                    col = self._get_col_with_log(df_cols, k, sheet)
                    return _clean(row.get(col)) if col and pd.notna(row.get(col)) else None

                def _bit(k):
                    col = self._get_col_with_log(df_cols, k, sheet)
                    v = str(row.get(col)).strip().upper() if col else ''
                    return 1 if v in ['1', '1.0', 'TRUE', 'EVET', 'AKTIF'] else 0

                p_no = _val('pc_no')
                s_no = _val('seri_no')
                ip = _val('ip')
                m_id = _val('mahal_id')

                if not any([p_no, s_no, ip, m_id]): continue
                final_pc_no = p_no or s_no or ip or f"AUTO-{dev_type}-{index+2}"

                payload = (
                    dev_type, final_pc_no, s_no, _val('monitor_seri'), m_id, ip, _val('mac'),
                    _bit('rdp'), _bit('depo'), _bit('arizali'), _bit('mahalsiz'), _bit('bekliyor'),
                    _val('yazicilar'), _val('phone'), _val('aciklama')
                )

                existing = conn.execute("SELECT id FROM inventory WHERE pc_no=? OR (seri_no=? AND se
                if existing:
                    conn.execute("UPDATE inventory SET device_type=?, pc_no=?, seri_no=?, monitor_se
                else:
                    conn.execute("INSERT INTO inventory (device_type, pc_no, seri_no, monitor_seri,
                    stats[dev_type] += 1
            except Exception as e:
                logger.error(f"Satir {index+2} Hatasi: {e}")
            conn.commit()

        stats["total"] = sum(v for k,v in stats.items() if k != "total")
        return stats
    finally:
        conn.close()

def sync_service_records(self):
    logger.info("=== SERVIS KAYITLARI SENKRONIZASYONU BASLATILDI ===")
    # KURAL: Servis Excel dosyası da ana_database içinde aranıyor
    path = os.path.join(self.db_folder, 'servise_giden_yazicilar.xlsx')
    if not os.path.exists(path):

```

```

        logger.warning(f"Servis Exceli bulunamadi, atlaniyor: {path}")
        return

conn = get_db_connection()
try:
    df = pd.read_excel(path)
    df.columns = df.columns.str.strip().str.lower()
    logger.info("Servis kayitlari isleniyor...")
    # ... (Diger isleme mantigi)
    conn.commit()
except Exception as e:
    logger.error(f"Servis Sync Hatasi: {e}")
finally:
    conn.close()

```

Dosya: core/database_sql.py

```

import sqlite3
import os
from dotenv import load_dotenv

load_dotenv()

# Veritabanı yolu
DB_PATH = os.getenv('DB_PATH', 'database/inventory.db')

def get_db_connection():
    # Yetki hatasını engellemek için klasör oluşturma işlemini try-except içine alıyoruz
    db_dir = os.path.dirname(DB_PATH)
    try:
        if db_dir and not os.path.exists(db_dir):
            os.makedirs(db_dir, exist_ok=True)
    except Exception as e:
        print(f"UYARI: Database klasörü oluşturulamadı (Zaten var olabilir veya yetki yok): {e}")

    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db_connection()
    try:
        # 1. Locations Table
        conn.execute('''CREATE TABLE IF NOT EXISTS locations (
            mahal_kodu TEXT PRIMARY KEY,
            mahal_adi TEXT NOT NULL,
            phone TEXT,
            updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
        )''')

        # 2. Inventory Table
        conn.execute('''CREATE TABLE IF NOT EXISTS inventory (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            device_type TEXT NOT NULL,
            pc_no TEXT UNIQUE,
            seri_no TEXT,
            monitor_seri TEXT,
            mahal_id TEXT,
            ip TEXT,
            mac TEXT,
            is_rdp INTEGER DEFAULT 0,
            is_depo INTEGER DEFAULT 0,
            is_arizali INTEGER DEFAULT 0,
            is_mahalsiz INTEGER DEFAULT 0,
            is_kurulum_bekliyor INTEGER DEFAULT 0,

```

```

        bagli_yazicilar TEXT,
        phone TEXT,
        aciklama TEXT,
        updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
    )'''

    # Diğer tablolar...
    conn.execute("CREATE TABLE IF NOT EXISTS audit_logs (id INTEGER PRIMARY KEY AUTOINCREMENT, table_name TEXT, user TEXT, action TEXT, updated_at DATETIME DEFAULT CURRENT_TIMESTAMP)")

    conn.commit()
finally:
    conn.close()

def query_db(query, args=(), one=False):
    conn = get_db_connection()
    cur = conn.execute(query, args)
    rv = cur.fetchall()
    conn.close()
    return (rv[0] if rv else None) if one else rv

```